Assignment II (Set A - Group 1 - 100)

Robust Reinforcement Learning under Sticky Actuators

Total Marks = 13 Marks

Intended Learning Outcome:

Students should be able to

- Understand the basic functionality of the Q-learning (off-policy) and SARSA (on-policy) temporal-difference control methods.
- Implement Q-learning and SARSA with state discretization, and compare off-policy and on-policy learning under stochastic action execution.

Prerequisite:

- Students should go through the lectures CS7 – CS12.
- Webinar demonstrations.

Please note that this assignment will involve some amount of self-learning (on the part of modelling solutions appropriately + programming skills).

Submission Deadline: [November 14, 2026]

Instructions

- Read the assignment proposal carefully.
- Solve all the assignment problems. Submit only one solution file (Group # - Q_learning_SARSA_Set A).
- It is mandatory to submit the assignment in **PDF format**, consisting of all the codes and outputs with each and every iteration. Any other format will not be accepted.
- Only one submission per group is required.
- Add comments and descriptions to every function you create and every operation you perform. If not found, 1 mark will be deducted. Comments help the evaluator understand your code quickly and clearly.
- **Group Contribution Declaration:** At the beginning of the report, list the names of all group members and the percentage contribution of each member.
- **No last-moment or email submissions will be accepted**. Submit the first version of your assignment 2 days before the deadline.
- **The assignment must be executed in the virtual lab**, and **screenshots with timestamps from the virtual lab must be attached** in the final submission.
- Plagiarized submissions will not be evaluated at both the parties.
- Key factors considered for scoring include (but are not limited to): Creativity, Originality, Uniqueness, Code Modularity, Representation, Reproducibility, Similarity to AI tool solutions.

How to reach out for clarifications

This assignment is administered by:

1) Sandeep – sandeep.yadav@wilp.bits-pilani.ac.in (From group 1 to 40)
2) Meet Shah – meetshah@wilp.bits-pilani.ac.in (From group 41 to 80)
3) Maneesha Rai – raimaneesha@wilp.bits-pilani.ac.in (From group 81 to 120)

1) Subash – subasharun@wilp.bits-pilani.ac.in (From group 121 to 160)
2) Divya KalyanKumar – divyak@wilp.bits-pilani.ac.in (From group 161 to 200)

Any request for clarification must be addressed through email (official email only) to all five instructors listed.

Messaging in TEAMS is discouraged, so that all transactions are tracked. Clarifications relevant to all students will be shared through the discussion forums.

Question

The **MountainCar-v0** environment in Gymnasium consists of:

- **State Space:** A 2-dimensional continuous state vector: car position in [−1.2, 0.6] and car velocity in [−0.07, 0.07].
- **Action Space:**

Action	Description
0	Accelerate to the left
1	Do not accelerate
2	Accelerate to the right

The base environment gives a reward of −1 per time step. An episode terminates when the car reaches the flag (position ≥ 0.5) and is truncated after 200 steps.

In practical control systems, actuators do not always respond immediately to new commands. A motor controller may keep applying the previous command because of latency, a stuck relay or a slow control loop. In this assignment, investigate the effect of **sticky (delayed) actuators** on reinforcement learning by creating a modified version of MountainCar-v0 and comparing a **tabular Q-learning** agent with a **tabular SARSA** agent.

Create a custom environment by wrapping the original **MountainCar-v0** environment using **gym.Wrapper**. The wrapper must modify the **action execution process** and the **reward function** while keeping all other aspects of the original environment unchanged.

Environment Modifications

The wrapper must keep one internal variable, **a_prev**, the action that was **actually executed** on the previous step. On every call to **reset()**, set a_prev = 1 (Do not accelerate). For every call to **step(action)**, the wrapper must perform the following operations in the given order.

Step 1: Receive the Agent's Action

Let **a** denote the action selected by the agent. Store this action before applying any modification. It will later be used to compute the modified reward.

Step 2: Simulate a Sticky Actuator

Before passing the action to the original environment, modify it according to the following rules.

- If **a = a_prev**, execute the action without modification.
- If **a ≠ a_prev**:
  - Generate a random number **r** from a uniform distribution in the interval **[0, 1)**.
  - If **r < 0.20**, the actuator is stuck: replace the selected action with **a_prev**.
  - Otherwise, execute the originally selected action.

Let **a_exec** denote the action finally executed. After execution, update **a_prev = a_exec**.

The agent **must not receive any indication** that its selected action was replaced.

Example (episode just reset, so a_prev starts at 1)

Step	a_prev	Agent Action (a)	Random Number	Executed Action (a_exec)
1	1	2	0.55	2
2	2	2	–	2
3	2	0	0.11	2
4	2	0	0.74	0
5	0	1	0.05	0

Step 3: Execute the Action

- Execute **a_exec** in the original MountainCar-v0 environment.
- Receive the following outputs from the base environment: observation, base reward (R_base), terminated, truncated, info.

Step 4: Compute the Modified Reward

Replace the reward returned by the original environment with

$$R = R_base - 0.2 \times \mathbb{1}[a \neq 1] + B$$

where

- R_base is the reward returned by the original environment (−1 per step).
- **a** is the **action originally selected by the agent** (before the sticky-actuator simulation).
- $\mathbb{1}[a \neq 1]$ is an indicator function that equals **1** when the selected action is **0** or **2**, and **0** otherwise.
- B is the gentle-arrival bonus defined in Step 5.

Note: The energy penalty depends on the **action selected by the agent**, **not** on the action actually executed. Therefore:

- if the agent selects a push action (0 or 2) and it is executed, subtract **0.2**;
- if the agent selects a push action but the actuator is stuck on another action, subtract **0.2**;
- if the agent selects **Do not accelerate** (1), no penalty is applied, **even if** the stuck actuator keeps pushing.

Step 5: Gentle-Arrival Bonus

Award the bonus **B = +20** only if all of the following conditions are satisfied simultaneously:

- terminated == True
- truncated == False
- observation[0] ≥ 0.5 (car has reached the flag)
- observation[1] ≤ 0.04 (car arrives at a controlled speed, not at full speed)

Observation Index	Description
observation[0]	Car position
observation[1]	Car velocity

If any condition is not satisfied, including truncation at 200 steps or arriving at the flag faster than 0.04, set **B = 0**.

Step 6: Return the Environment Output

Return observation, modified reward (R), terminated, truncated and info to the agent. No information about stuck actions or reward modifications may be added to the returned info dictionary.

Components That Must Remain Unchanged

Except for the modifications described above:

- the observation space must remain unchanged;
- the action space must remain unchanged;
- the state-transition dynamics must remain unchanged (except for the sticky-action replacement);
- the episode termination and truncation conditions must remain unchanged; and
- no additional observations or state variables may be exposed to the agent.

Tasks

(a) Modified Environment Implementation and Verification (2 Marks)

Implement the modified MountainCar-v0 environment exactly as specified in the **Environment Modifications** section.

Verify the correctness of your implementation by executing multiple episodes using a random policy. Demonstrate experimentally that:

- Approximately **20%** of all attempted action changes (steps where a ≠ a_prev) are replaced by the previously executed action.
- No replacement ever occurs when the agent repeats the previously executed action.
- The energy penalty is applied for **every selected push action** (0 or 2), irrespective of the action actually executed, and is **never** applied when the agent selects action 1.
- The **+20** bonus is awarded **only** when the gentle-arrival criterion is satisfied.

Present appropriate statistics, logs or experimental evidence to support your verification. Since a random policy rarely reaches the flag, you may additionally use a simple hand-coded policy (for example, push in the direction of the velocity) to verify the bonus.

(b) Q-Learning (3 Marks)

Implement and train a **tabular Q-learning** agent consisting of:

- State discretization: divide position and velocity into uniform bins (at least **20 × 20**; justify your choice).
- A Q-table of size (position bins × velocity bins × 3), initialised to zero.
- An **ε-greedy** exploration strategy with a decay schedule of your choice (state it clearly).
- The off-policy update rule:

$$Q(s, a) \leftarrow Q(s, a) + \alpha\,[\,R + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a)\,]$$

Here a is the action **selected by the agent**, since the executed action is hidden from it. Do not bootstrap from s' when the episode is terminated (truncation is not a terminal state).

Train the agent on:

- the **modified** MountainCar-v0 environment; and

- the **original** MountainCar-v0 environment,

using the **same random seed, discretization, learning rate α, discount factor γ, exploration schedule and number of training episodes** (at least **5,000**) for both experiments.

(c) SARSA (3 Marks)

Implement and train a **tabular SARSA** agent.

- Use the **same discretization, Q-table initialisation, α, γ, exploration schedule, random seed and number of training episodes** as the Q-learning implementation.
- The **only difference** between the two implementations must be the TD target. SARSA uses the next action a' actually chosen by the ε-greedy policy:

$$Q(s, a) \leftarrow Q(s, a) + \alpha\,[\,R + \gamma \cdot Q(s', a') - Q(s, a)\,]$$

- Train the SARSA agent on both the **modified** and **original** environments under identical experimental conditions.

(d) Performance Evaluation (3 Marks)

Compare the following four trained agents:

- Q-learning – Original Environment
- SARSA – Original Environment
- Q-learning – Modified Environment
- SARSA – Modified Environment

Plot and analyse the following:

- **Episode reward versus training episode** (with a 100-episode moving average).
- **Average predicted value, max over a of Q(s, a), versus training episode**, computed on a **fixed validation set of states** sampled once before training and unchanged throughout.
- **Success rate (reaching the flag) and gentle-arrival rate versus training episode**, each a moving average over the previous 100 episodes.
- **Average number of push actions (0 or 2) selected per episode versus training episode.**
- **Greedy policy map**: a heatmap of the greedy action over the position × velocity grid for each of the four agents.

Also evaluate each final greedy policy (ε = 0) over **100 test episodes** with fixed seeds, and report mean reward, success rate, gentle-arrival rate and mean episode length in one table.

Provide a brief interpretation of each plot, highlighting the differences between Q-learning and SARSA and between the original and modified environments.

(e) Discussion (2 Marks)

Based on your experimental results, answer the following questions.

- Does the sticky actuator increase the difference between the value estimates of Q-learning and SARSA? Justify your answer using the Q-value plots.
- The agent cannot observe a_prev. Why does this make the modified environment non-Markovian with respect to the observation, and how does it affect credit assignment?
- Do the energy penalty and the gentle-arrival bonus encourage fewer pushes or a slower, more controlled arrival? Support your answer using the push-count plot, the policy maps and the arrival-speed statistics.
- Which algorithm performs better under sticky actuators? Is this consistent with the theoretical difference between on-policy (SARSA) and off-policy (Q-learning) control? Explain.
- Identify one limitation of your experimental setup and suggest one possible improvement.